

**ШИНЖЛЭХ УХААН, ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ**



БИЕ ДААЛТЫН ТӨСӨЛ

Программчлалын дадлага F.CSM360

Сэдэв: Cyberpunk Z-Type бичих хурдны тоглоом

Хичээл заасан багш:

Ж.ОРГИЛ /F.CS08/

Гүйцэтгэсэн:

Б. Энхдөлгөөн /B251960012/

Б. Хүслэн /B251960043/

Улаанбаатар хот

2026

1. ХУРААНГУЙ (ABSTRACT)

Энэхүү тайланд Python хэл болон Pygame сан дээр суурилсан, хэрэглэгчийн товчлуураар (touch typing) хурд болон нарийвчлалыг хөгжүүлэх зорилготой "Cyberpunk Z-Type" хэмээх интерактив тоглоомын системийн хөгжүүлэлтийн үр дүнг танилцуулна. Систем нь Англи, Монгол хэлний горимыг дэмжих бөгөөд дэлгэц дээгүүр хурдасч унах үгсийг тоглогч гараараа алдаагүй шивж устгах үндсэн механиктай. Зорилтот үгийг хайх болон сумыг (projectile) саатуулах логикт шугаман хайлт (linear search) болон координатын зайн алгоритмууд ашиглагдсан. Түүнчлэн системийн процессорын ачааллыг бууруулах үүднээс текстийн неон гэрэлтэлтийг (neon-glow) дахин зурахаас сэргийлж хэш хүснэгт (hash map) дээр суурилсан кэшлэх механизмыг хэрэгжүүлсэн. Гүйцэтгэлийн шинжилгээний үр дүнд, систем нь гол алгоритмуудын $O(N)$ хугацааны нийлмэлтэйгээр секундэд 60 удаа (60 FPS) гацалтгүй, тогтвортой шинэчлэгдэн ажиллаж байгааг баталсан болно.

2. ОНОЛЫН СУДАЛГАА (THEORETICAL RESEARCH)

2.1 Өгөгдлийн бүтэц (Data Structures)

Системийн хурд болон санах ойн ашиглалтыг оновчтой байлгах үүднээс дараах өгөгдлийн бүтцүүдийг ашигласан:

- **Динамик хүснэгт (Dynamic Array / Python List):** Тоглоом доторх хөдөлгөөнт объектууд болох words, projectiles, particles-ийг хадгалахад *LinkedList*-ийн оронд *Array*-г сонгосон. Хэдийгээр дунд хэсгээс элемент устгахад $O(n)$ хугацаа шаардах боловч дэлгэц зурах (draw) болон шинэчлэх (update) үйлдэл нь объектуудаар дараалан гүйх шаардлагатай байдаг тул санах ойд дараалж байрладаг хүснэгт нь процессорын кэшд илүү үр ашигтай ажилладаг.
- **Хэш хүснэгт (Hash Map / Python Dictionary):** Текстэнд олон давхар Surface ашиглан неон эффект үүсгэх нь тооцооллын өртөг өндөртэй байдаг. Үүнийг шийдвэрлэхийн тулд `_neon_cache` толь бичигт урьдчилан зурагдсан гадаргууг түлхүүр утгаар хадгалж, дараагийн дуудалтад дунджаар $O(1)$ хугацаанд шууд буцаах (memoization) аргыг ашигласан.
- **Олонлог (Set):** Тухайн түвшинд нэг үг давхардаж гарахаас сэргийлэх зорилгоор `used_words_this_level` олонлогийг ашигласан. Олонлог нь өгөгдлийн гишүүнч-

лэлийг дунджаар $\mathcal{O}(1)$ хугацаанд шалгадаг тул жагсаалтаар хайлт хийснээс хамаагүй өндөр хурдтай.

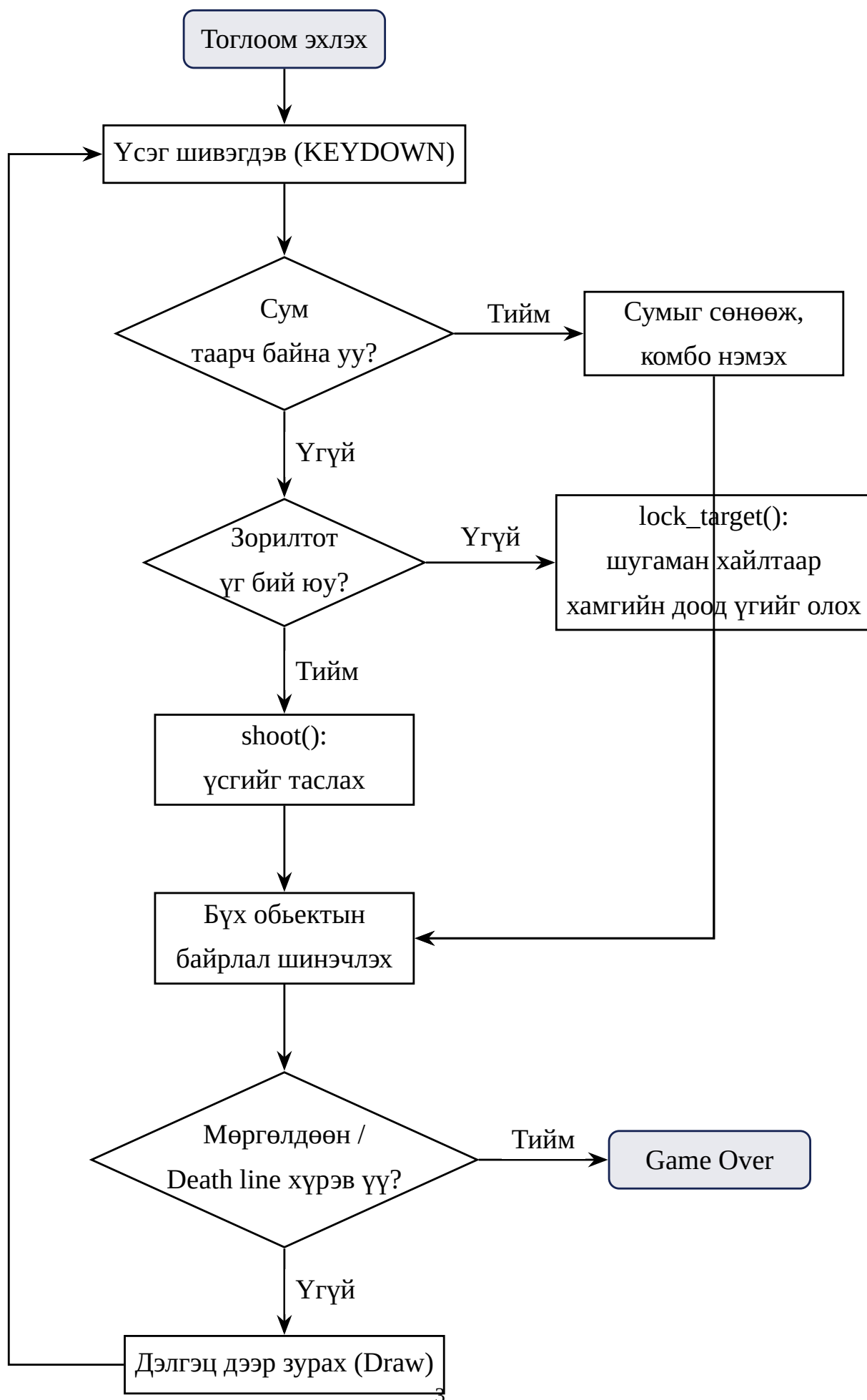
2.2 Сонгодог алгоритмууд (Classic Algorithms)

- **Шугаман хайлт (Linear Search):** Тоглогч үсэг дарах бүрт дэлгэц дээрх идэвхтэй үгсийг шалгаж, тохирох үгсийг шүүдэг. Объектүүдийн тоо бага ($W < 20$) бөгөөд эрэмбэлэгдээгүй байдаг тул Хоёртын хайлт (Binary Search) ашиглах шаардлагагүй бөгөөд $\mathcal{O}(W)$ хугацааны шугаман хайлт нь хамгийн оновчтой шийдэл юм.
- **Координатын геометрийн мөргөлдөөн (Euclidean Collision):** Дайсны сум хөлгийг мөргөсөн эсэхийг Пифагорын теоремоор буюу хоёр цэгийн хоорондох зайгаар хэмждэг ($d = \sqrt{(x_p - x_s)^2 + (y_p - y_s)^2} \leq R$). Алгоритм нь сум бүрийн хувьд $\mathcal{O}(1)$ хугацаанд шалгагдана.
- **Шугаман интерполяци (Linear Interpolation, Lerp):** Дэлгэц дээрх оноо, WPM, болон явцын хувийг огцом солихын оронд $a + (b - a) \cdot t$ томъёогоор алхам алхмаар жигдрүүлж шилжүүлдэг нь интерфэйсийг (UI) илүү уян хатан харагдуулна.

3. АЛГОРИТМЫН ЗОХИОМЖ (ALGORITHM DESIGN)

3.1 Үйл ажиллагааны диаграмм (Flowchart)

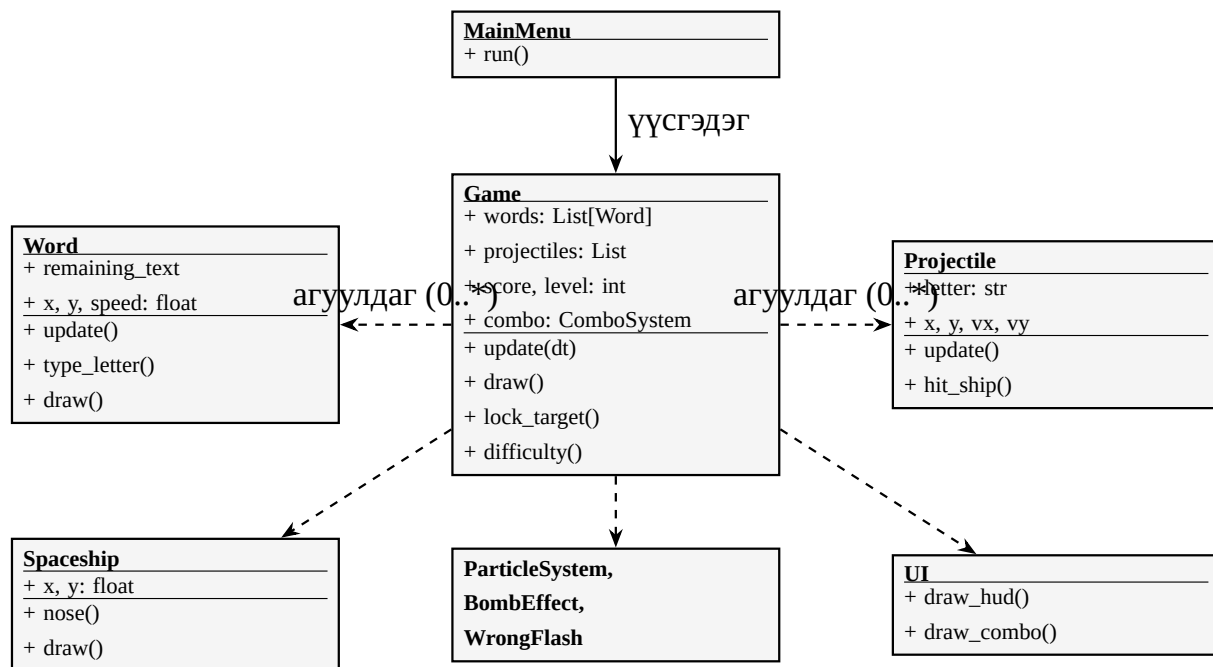
Тоглоомын гол гогцоо нь оролт унших, төлөв шинэчлэх, зурах гэсэн үндсэн гурван циклийг секундэд 60 удаа гүйцэтгэдэг.



Зураг 1: Тоглоомын гол гогцооны үйл ажиллагааны диаграмм

3.2 Класс диаграмм (UML Class Diagram)

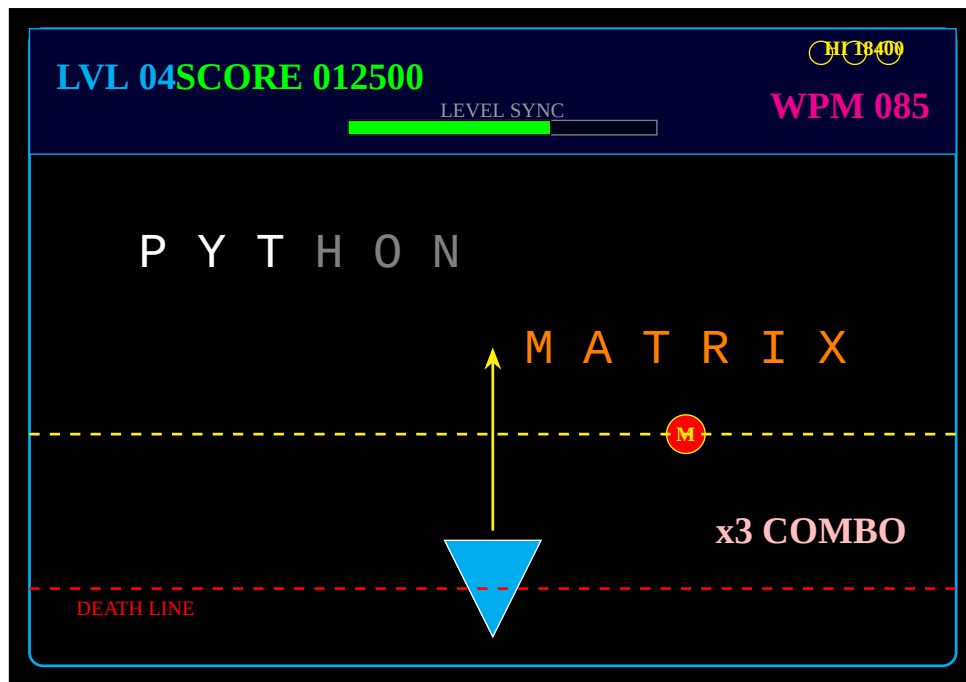
Системийн архитектурыг Объект хандлагат программчлалын (ООР) дагуу зохион байгуулсан. Блокуудын давхардалтаас сэргийлж координатыг нарийвчлан зааж өглөө.



Зураг 2: Объектүүдийн ассоциаци (Association) холбоо

3.3 Дэлгэцийн зохиомж (UI Prototype)

Хэрэглэгчийн интерфэйсийг (UI) тоглоомын киберпанк хэв маягт тохируулан бүтээсэн нооргийг доор харуулав.



Зураг 3: Тоглоомын дэлгэцийн зохиомжийн (HUD) ноорог

3.4 Псевдо алгоритм (Pseudocode)

Онилох болон буудах (lock_target, shoot) гол логикийг дараах байдлаар боловсруулсан.

```

1 Function Handle_Typed_Letter(letter):
2     If a flying projectile matches letter:
3         Destroy nearest (lowest) matching projectile
4         Increase combo, score, spawn laser + particles
5         Return
6
7     If no target word is currently locked:
8         candidates = [w for w in active_words if w.first_char ==
9             letter]
9         If candidates is empty:
10             Reset_Combo()
11             Trigger_Wrong_Flash()
12         Else:
13             target = word with maximum Y among candidates
14             target.locked = True
15             Shoot(letter)
16     Else:
17         Shoot(letter)

```

```
18
19 Function Shoot(letter):
20     If target.first_char != letter:
21         Reset_Combo()
22         Trigger_Wrong_Flash()
23         Return
24     Remove first character from target
25     Increase combo streak, score, counters
26     Spawn laser beam + particles
27     If target has no characters left:
28         Destroy target word, check level-up
29         target = NULL
```

Listing 1: Зорилтот үг тогтоох болон үсэг таслах логик

4. ҮР ДҮН (RESULTS)

4.1 Нийлмэл байдлын шинжилгээ (Complexity Analysis)

Зохиосон алгоритмуудын онолын үнэлгээг (Big- $\mathcal{O}$ notation) дараах байдлаар тооцоолсон:

- **Зорилтот үг тогтоох / Сум таслах (lock_target):** Хугацааны нийлмэл нь $\mathcal{O}(W + P)$ байна. Энд W нь идэвхтэй унаж буй үгийн тоо, P нь сумны тоо бөгөөд практик дээр хоёулаа 20-оос хэтрэхгүй тул бодит цагийн тооцоололд ачаалал үүсгэхгүй.
- **Дэлгэц шинэчлэх (update):** Гогцоо бүрд объект бүрийн байрлалыг шинэчлэх бөгөөд нийлмэл нь $\mathcal{O}(W + P + E)$ байна (E : идэвхтэй бөөмсийн тоо). Энэхүү шугаман өсөлт нь секундэд 60 удаа зурагдах FPS-ийг тогтвортой барьж чадна.
- **Неон текст зурах (draw_neon_text):** Хэш кэш ашигласан тул өмнө нь зурагдсан үгийг санах ойгоос унших хугацааны нийлмэл дунджаар $\mathcal{O}(1)$ байна. Харин санах ойн нийлмэл нь дэлгэцэд гарсан ялгаатай үг болон өнгөний хослолоос хамаарч $\mathcal{O}(C)$ хэмжээгээр нэмэгдэнэ.

4.2 Ажиллагааны бодит үр дүн

Тоглоомын механик нь дизайны шаардлагыг бүрэн хангаж ажиллаж байна.

1. **Эхлэл цэс (MainMenu):** Англи болон Монгол хэлний сонголт хэвийн ажиллах бөгөөд хамгийн өндөр онооны (`highscore.json`) бүртгэл автоматаар уншигдана.
2. **Үндсэн тоглолт:** Үгийн эхний үсгийг зөв дармагц лазерын шугам үүсэж, бөөмс (`particles`) цацагдах зэрэг график эффектүүд амжилттай хэрэгжсэн. Буруу үсэг дарахад дэлгэцийн ирмэг улаанаар анивчих (`WrongFlash`) үйлдэл хэвийн ажиллаж байна.
3. **Тоглоомын хүндрэл:** 4-р түвшингээс эхлэн дайсны үгнүүд хөлөг рүү буудаж (`Projectile`) эхлэх бөгөөд тоглогч тэдгээрийг шивж устгах логик амжилттай туршигдсан.

5. ДҮГНЭЛТ (CONCLUSION)

Нэгтгэл: Энэхүү бие даалтын төслийн хүрээнд "Cyberpunk Z-Type" программыг Python хэл, Pygame санг ашиглан амжилттай хөгжүүллээ. Систем нь Объект Хандлагат архитектурт бүрэн суурилсан бөгөөд динамик хүснэгт, хэш кэш, олонлог зэрэг өгөгдлийн бүтцийг зориулалтын дагуу зөв хослуулсны үр дүнд тооцооллын өндөр хурдтайгаар ажиллаж чадаж байна.

Бэрхшээл ба Сургамж: Төслийг хөгжүүлэх явцад неон гэрэлтэлттэй (`glow`) олон тооны текстийг дэлгэц дээр секунд бүр давтан зурахад FPS унах бэрхшээл тулгарсан. Үүнийг шийдвэрлэхийн тулд `_neon_cache` хэш хүснэгтэд суурилсан Memoization арга хэрэглэснээр гүйцэтгэлийг үлэмж хэмжээгээр сайжруулж чадсан нь өгөгдлийн бүтцийн зөв сонголт хэрхэн чухал болохыг практик дээр баталлаа. Мөн хүндрэлийн түвшнийг математик функцээр (`difficulty()`) удирдах нь кодыг цэвэр бөгөөд өргөжүүлэх боломжтой болгодгийг энэхүү төслөөс суралцсан болно.

6. АШИГЛАСАН МАТЕРИАЛЫН ЖАГСААЛТ (REFERENCES)

1. M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Python*. Wiley, 2013.
2. *Pygame Documentation*. Pygame Community. [Цахим эх сурвалж]. Хаяг: <https://www.pygame.org/docs/>
3. Python Software Foundation. *Python 3 Object-Oriented Programming*. [Цахим эх сурвалж]. Хаяг: <https://docs.python.org/3/tutorial/classes.html>

4. *wonderwords* -- Python random word generation library. [Цахим эх сурвалж]. Хаяг: <https://pypi.org/project/wonderwords/>
5. R. Nystrom, *Game Programming Patterns*, Genever Benning, 2014. (Архитектур, Game Loop болон Object Pooling-ийн онол)
6. Python *collections* модулийн баримтжуулалт. Хаяг: <https://docs.python.org/3/library/collections.html>